



**National University of Sciences and Technology (NUST)**  
**School of Electrical Engineering and Computer Science**

**Faculty of Computing**

**CS 212: Object-Oriented Programming**

**Lab 08: Polymorphism**



## Lab 08: Polymorphism

---

In this lab, you will build an object-oriented application to model a hospital system. You will use Java to apply polymorphism, encapsulation, and inheritance. Below are the clear instructions and tasks you need to follow to complete this assignment. Each step builds on the previous one, so please read and follow carefully.

### 1. Design the Hospital Structure

You need to design the basic structure of the hospital. This is the first step, where you define the hospital's attributes.

Attributes of the hospital:

- Hospital Name: The name of the hospital (e.g., 'City Hospital').
- Address: Where the hospital is located.
- Departments: A hospital has different departments (e.g., Surgery, Pediatrics, etc.).
- Patients: You need a way to store patients in the hospital.

### 2. Create Departments and Team Members

Each department in the hospital has a team of members, which include doctors and nurses.

Attributes of a department:

- Department Name: The name of the department (e.g., Surgery, Emergency).
- Staff (Team Members): This is a list of doctors and nurses in that department.

Team members (doctors and nurses) have the following attributes:

- Name: The name of the team member.
- ID: A unique identification number.
- Gender: Male or Female.
- Date Joined: When they joined the hospital.
- Working Hours: Each team member works a maximum of 12 hours per day.

### 3. Different Types of Doctors

Doctors can be of three types:

1. Interns: Doctors who are still learning under a senior doctor's supervision.



## National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

2. Senior Doctors: Experienced doctors who supervise interns and treat patients.
3. Surgeons: Senior doctors who perform surgeries and manage their own group of patients.

Each doctor should have:

- Specialty: What type of doctor they are (e.g., heart specialist, surgeon).
- Supervisor: Interns should have a senior doctor as their supervisor.

### **4. Add and Remove Patients and Staff**

The system should allow adding and removing both patients and team members (doctors/nurses) from the hospital or specific departments.

Patients have the following attributes:

- Name
- Birth Date
- Gender
- Date Accepted (When the patient entered the hospital)
- Diagnosis Report
- Treating Doctor
- Number of Days Stayed

### **5. Implement Patient Treatment Method**

Doctors treat patients, but interns, senior doctors, and surgeons all treat patients in different ways. You need to write a method that models how each type of doctor treats a patient differently.

- Interns will follow the instructions of a senior doctor.
- Senior Doctors will have their own way of treating patients.
- Surgeons have a specialized method for treating patients, especially in the case of surgeries.

Each doctor should have its own implementation of the method ``treatPatient()``.

### **6. Patient Reports**

Each doctor should be able to check a specific patient's report.

Write a method that allows doctors to access the patient's report and view details such as the diagnosis and the number of days the patient has been admitted.



## **7. Use of Encapsulation**

All data fields (e.g., patient name, doctor specialty) should be encapsulated. This means:

- Use private fields and provide getter and setter methods to access and modify them.

For example, to access the name of a doctor, use the method `getName()`.

### **Steps for Implementation:**

Step 1: Create the Hospital Class

- Define the hospital with attributes: `name`, `address`, and `departments`.
- Provide methods to add and remove patients and team members.

Step 2: Create the Department Class

- Define the department with `name` and a list of team members (doctors and nurses).
- Implement methods to add/remove team members.

Step 3: Create the Team Member Class (Abstract Class)

- Define common attributes for team members (`name`, `ID`, `gender`, `date joined`, `working hours`).
- Include an abstract method `treatPatient()` that will be implemented by specific types of doctors (intern, senior doctor, surgeon).

Step 4: Create the Doctor Classes (Intern, SeniorDoctor, Surgeon)

- Each subclass should implement the `treatPatient()` method differently based on the type of doctor.
- Interns should treat patients under the supervision of senior doctors.

Step 5: Create the Patient Class

- Define attributes: `name`, `birth date`, `gender`, `date accepted`, `diagnosis report`, `treating doctor`, and `days stayed`.
- Provide methods to view and update patient reports.

Step 6: Create Methods to Add/Remove Patients and Doctors

- The hospital should have methods to add and remove patients and team members (doctors and nurses).

Step 7: Test the Application

- After completing the implementation, write a main method to test all the functionality:
  - Add patients and doctors to the hospital.
  - Simulate doctors treating patients and checking their reports.



### Expected Output:

```
Dr. John (Intern) is treating the patient under supervision of Dr. Smith
Dr. Smith (Senior Doctor) is treating the patient
Dr. Lee (Surgeon) is performing surgery on the patient
Patient Name: John Doe
Diagnosis Report: Flu
Doctor: Dr. Smith
Days in Hospital: 5
Hospital: City Hospital, Address: 123 Main St
Departments:
Surgery
Pediatrics
```